Análise Empírica de Algoritmos

Ajuste por razão consecutiva e razão dobrando

Determinação Empírica de Constantes de Complexidade

Modelo de Caixa-Preta (Forma Básica)

Dado um algoritmo com tempo de execução assintótico $\Theta(b^{an} \cdot n^d \cdot \log^e n)$, modelamos as medições experimentais de tempo $T(n)$ por:

$$T(n) \approx c \cdot (b^a)^n \cdot n^d \cdot \log^e n$$

onde $c > 0$ é a constante oculta pela notação assintótica e n é o tamanho da entrada.

- **Acoplamento das Constantes a e b:**
  - Como $b^{an} = (b^a)^n$, a observação externa do tempo não permite separar a e b.
  - Vamos então fixar $b = 2$, e estimar o valor de a.
  - Vamos também assumir que o *log* tem base 2.
- **Objetivo:** Determinar os parâmetros c, a, d e e a partir de uma tabela de coletas $\{(n_1, T(n_1)), (n_2, T(n_2)), \ldots, (n_m, T(n_m))\}$.

Estratégia 2: Razão consecutiva assintótica (com fator exponencial)

Isolamento da Base Exponencial ($2^a > 1$)

Se o algoritmo possui crescimento exponencial, o fator $(2^a)^n$ domina completamente para valores grandes de n. Avaliando a razão entre entradas consecutivas:

$$\lim_{n\to\infty} \frac{T(n+1)}{T(n)} = 2^a.$$

- **Procedimento Prático:**
  1. Coletar tempos para $n, n+1, n+2, \ldots$ no limite de viabilidade do hardware.
  2. A razão $T(n+1)/T(n)$ converge diretamente para a base efetiva 2^a.

Cálculo da Constante c (após estimar d e e na estratégia seguinte)

$$c \approx \frac{T(n)}{2^a \cdot n^d \cdot \log^e n} \quad \text{(melhor usar o maior } n \text{ disponível).}$$

Estratégia 2: Razão consecutiva assintótica (com fator exponencial)

$$\begin{aligned}
\lim_{n\to\infty} \frac{T(n+1)}{T(n)} &= \lim_{n\to\infty} \frac{c\cdot (2^a)^{n+1}\cdot (n+1)^d \cdot \log^e(n+1)}{c\cdot (2^a)^n \cdot n^d \cdot \log^e n} \\
&= \lim_{n\to\infty} 2^a \cdot \left(1+\frac{1}{n}\right)^d \cdot \left(\frac{\log n + \log(1+1/n)}{\log n}\right)^e \\
&= 2^a \cdot \lim_{n\to\infty} \left(1+\frac{\log(1+1/n)}{\log n}\right) = 2^a.
\end{aligned}$$

Estratégia 3: Razão dobrando assintótica (sem fator exponencial)

Quando não há termo exponencial ($b^a = 1$) ou após dividir $T(n)$ pelo fator $(b^a)^n$ já obtido, temos $T(n) \approx c \cdot n^d \cdot \log^e n$. Dobrando a entrada e analisando o quociente:

$$\frac{T(2n)}{T(n)} \approx \frac{c \cdot (2n)^d \cdot \log^e(2n)}{c \cdot n^d \cdot \log^e n} = 2^d \cdot \left(\frac{1 + \log n}{\log n}\right)^e = 2^d$$

Caso sem a parte exponencial ($b^a = 1$)

$$\lim_{n \to \infty} \frac{T(2n)}{T(n)} = 2^d.$$

Obs.: Sem o fator exponencial, $T(n+1)/T(n)$ converge para 1 (não ajuda).
Serve apenas para determinar que o algoritmo não é exponencial.

Estratégia 3: Razão dobrando assintótica (sem fator exponencial)

Geralmente assumimos que $e = 0$, pois

- Estimativa confiável exige valores de n muito grandes, com tempo de execução proibitivo.

Cálculo da Constante c (após encontrar d e e)

$$c \approx \frac{T(n)}{n^d \log^e n} \approx \frac{T(n)}{n^d} \quad \text{(melhor usar o maior } n \text{ disponível).}$$

Exemplo Resolvido: Análise de Algoritmo Caixa-Preta

Um algoritmo desconhecido foi executado em ambiente controlado, gerando as seguintes medições de tempo de execução $T(n)$:

n	10	11	12	13
$T(n)$ **(segundos)**	3.07×10^{-2}	6.14×10^{-2}	1.23×10^{-1}	2.46×10^{-1}

Problema

Sabendo que a complexidade do algoritmo é da forma $\Theta(b^{an} \cdot n^d \cdot \log^e n)$:

1. Determine a base exponencial efetiva 2^a do algoritmo.
2. Determine a constante multiplicativa c assumindo $d = 0$ e $e = 0$.

Gabarito: Exemplo Resolvido

1. Determinação da Base Exponencial 2^a

Calculando a razão de tempos para valores sucessivos de n:

$$\frac{T(11)}{T(10)} = \frac{6.14 \times 10^{-2}}{3.07 \times 10^{-2}} = 2.00, \quad \frac{T(12)}{T(11)} = \frac{1.23 \times 10^{-1}}{6.14 \times 10^{-2}} \approx 2.00$$

$$\frac{T(13)}{T(12)} = \frac{2.46 \times 10^{-1}}{1.23 \times 10^{-1}} = 2.00 \implies \mathbf{2^a = 2}$$

2. Cálculo da constante c

- Como $d = 0$ e $e = 0$, temos $T(n) \approx c \cdot 2^n$. Para $n = 13$ (maior disponível):

$$c = \frac{T(13)}{2^{13}} = \frac{2.46 \times 10^{-1}}{1024} = 3.0 \times 10^{-5} \text{ s}$$

- **Modelo Empírico Final:** $T(n) \approx 3.0 \times 10^{-5} \cdot 2^n$ segundos.

Exercícios

Exercício: Estratégia 2 – Razão consecutiva

Um algoritmo exponencial foi submetido a testes para entradas consecutivas n, gerando as seguintes medições de tempo em segundos:

n	20	21	22	23
$T(n)$ **(s)**	0.1048	0.4398	1.8020	7.3680

Questão

Sabendo que $T(n) \approx c \cdot 2^{an} \cdot n^d \cdot \log_2^e n$:

1. Calcule as razões sucessivas $\frac{T(n+1)}{T(n)}$ para os valores da tabela.
2. Determine o valor limite aproximado da base 2^a e encontre a constante a.
3. Sabendo que $d = 1$ e $e = 0$, estime o valor da constante c usando os dados de $n = 23$.
4. Estime o tempo de execução para $n = 30$ usando o modelo obtido.

Gabarito: Estratégia 2 – Razão consecutiva

1. Cálculo das Razões Consecutivas

$$\frac{T(21)}{T(20)} = \frac{0.4398}{0.1048} \approx 4.196, \quad \frac{T(22)}{T(21)} = \frac{1.8020}{0.4398} \approx 4.097, \quad \frac{T(23)}{T(22)} = \frac{7.3680}{1.8020} \approx 4.088$$

2. Determinação de 2^a e de a

À medida que n cresce, os termos polinomiais e logarítmicos tendem a 1:

$$\lim_{n\to\infty} \frac{T(n+1)}{T(n)} = 2^a \approx 4.0 \implies 2^a = 2^2 \implies \mathbf{a = 2}$$

Gabarito: Estratégia 2 – Razão consecutiva

3. Cálculo da Constante c

Com $a = 2$, $d = 1$ e $e = 0$, temos $T(n) \approx c \cdot 2^{2n} \cdot n = c \cdot 4^n \cdot n$. Para $n = 23$:

$$c = \frac{T(23)}{4^{23} \cdot 23} \approx \mathbf{4.55 \times 10^{-15}} \text{ s}$$

4. Estimativa para $n = 30$

Usando o modelo $T(n) \approx 4.55 \times 10^{-15} \cdot 4^n \cdot n$, para $n = 30$:

$$T(30) \approx 4.55 \times 10^{-15} \cdot 4^{30} \cdot 30 \approx 1.57 \times 10^5 \text{ s} \approx 43.7 \text{ horas.}$$

Exercício: Estratégia 3 – Razão dobrando

Um algoritmo de grafos caixa-preta foi avaliado duplicando sucessivamente o tamanho da entrada n, obtendo os seguintes tempos:

n	1024	2048	4096
$T(n)$ **(s)**	0.5243	2.3069	10.0663

Questão

Assumindo o modelo assintótico $T(n) \approx c \cdot n^d$ (com $b^a = 1$ e $e = 0$):

1. Calcule as razões de duplicação $\frac{T(2048)}{T(1024)}$ e $\frac{T(4096)}{T(2048)}$.
2. Determine o grau polinomial d.
3. Calcule a constante multiplicativa c.
4. Estime o tempo de execução para $n = 10000$ usando o modelo obtido.

Gabarito: Estratégia 3 – Razão dobrando

1. Cálculo das Razões de Duplicação

$$\frac{T(2048)}{T(1024)} = \frac{2.3069}{0.5243} \approx 4.400, \quad \frac{T(4096)}{T(2048)} = \frac{10.0663}{2.3069} \approx 4.363$$

2. Grau Polinomial d

- Como as razões convergem para $4 = 2^2$ quando $n \to \infty$, temos $2^d = 4 \implies \mathbf{d = 2}$.

Gabarito: Estratégia 3 – Razão dobrando

3. Cálculo da Constante c

Para $n = 4096$ com $T(n) \approx c \cdot n^2$:

$$c = \frac{T(4096)}{4096^2} = \frac{10.0663}{16777216} \approx \mathbf{6.0 \times 10^{-7}}\ \text{s}$$

4. Estimativa para $n = 10000$

Usando o modelo $T(n) \approx 6.0 \times 10^{-7} \cdot n^2$, para $n = 10000$:

$$T(10000) \approx 6.0 \times 10^{-7} \cdot 10000^2 = 60\ \text{s}$$